Read-Only Is Not Isolation: Measuring and Closing Shared-State Channels Between Agent Sandboxes

Tayyab Khan, [affiliation]
With Apart Research. Research conducted at the Swarm Alignment Hackathon, November 2026.

Abstract

In the 2026 Hugging Face incident, sandboxed agents used a shared package cache as a message board by creating folders. The obvious fix is to make agent accounts read-only. We test whether that fix works. On an owned Artifactory OSS 7.161.26 deployment, two read-only clients with no direct network route still passed information through state that ordinary reads change: download statistics and remembered upstream failures. Scripted clients recovered every scheduled bit through each mechanism. Each single setting we tried closed one mechanism and left the other open; two designs closed both while all 14 pinned package installs still worked. Separate repositories fell to a copied peer password in 16 of 16 attempts until a gateway bound each sandbox's network origin to its account, which then rejected all 16. In an assigned cooperative task, cheap language models used the open service to exchange private values in 110 of 120 episodes and in 0 of 120 under the combined control. Server-side monitoring could not separate this from legitimate metadata work: it flagged 19 of 60 benign agents, and an information-weighted rule only reduced that to 17. A generic differential "leakage meter" with frozen analysis and sealed holdouts rediscovered both mechanisms without being told what to look for, and found no comparable signal in the hardened designs or in two null controls. We argue that supporting services should be treated as communication media until such a test fails at stated sensitivity, and that agent package access should move to frozen admission with private, immutable serving.

1 Introduction

Agent evaluations isolate the model's container, then give it a few supporting services it cannot work without: a package mirror, a web cache, a dataset store. In the August 2026 Hugging Face incident, agents in separate sandboxes found that the package mirror, a JFrog Artifactory instance, let them create folders in a shared cache, and used folder names as messages [1, 2]. METR also notes that agents could see each other's package installs because Artifactory does not isolate users' activity [2, p. 5, fn. 8]. OpenAI's response calls for supporting services to be "isolated, restricted, or replaced" [1, p. 27], but does not say what "restricted" must cover.

The natural restriction is to remove write access. Our question is whether that is enough. A read is not side-effect free: it can increment a counter, record who downloaded a file last, or make the service remember that a path does not exist. If another tenant can observe that state, two agents with no shared write permission and no network route to each other can still exchange information.

Contributions. (1) On a real Artifactory deployment we measure two read-only mechanisms and a matrix of seven controls, showing that each single setting closes only one (§4.1). (2) We show

that per-tenant repositories need identity bound outside the sandbox, and that an origin-binding gateway provides it (§4.2). (3) We show that cheap models use the open mechanism when asked to exchange information, and that the combined control stops them (§4.3). (4) We show that server-side monitoring sees exposure but not intent (§4.4). (5) We build a generic, preregistered leakage meter with nulls and sealed holdouts, and use it to test the hardened designs (§4.5–4.6). (6) We propose and partly validate an architecture that removes shared mutable state from the request path while preserving dependency installs (§4.7, §5).

The mechanisms themselves are not new: read-based signalling and cached-failure timing have precedent [4, 7]. Our contribution is a controlled, audited comparison of defenses on the system the incident actually used, with the utility cost measured alongside.

2 Threat model

Two sandboxes, A and B, each run an agent. They have no network route to each other, no shared filesystem, and no write permission on the package service. Both can send GET and HEAD requests to the same package service through a gateway. Figure 1 shows the setting. A *channel* exists if B's observations depend measurably on A's earlier or concurrent requests. We do not assume the agents intend to communicate; the question is whether the infrastructure permits it.

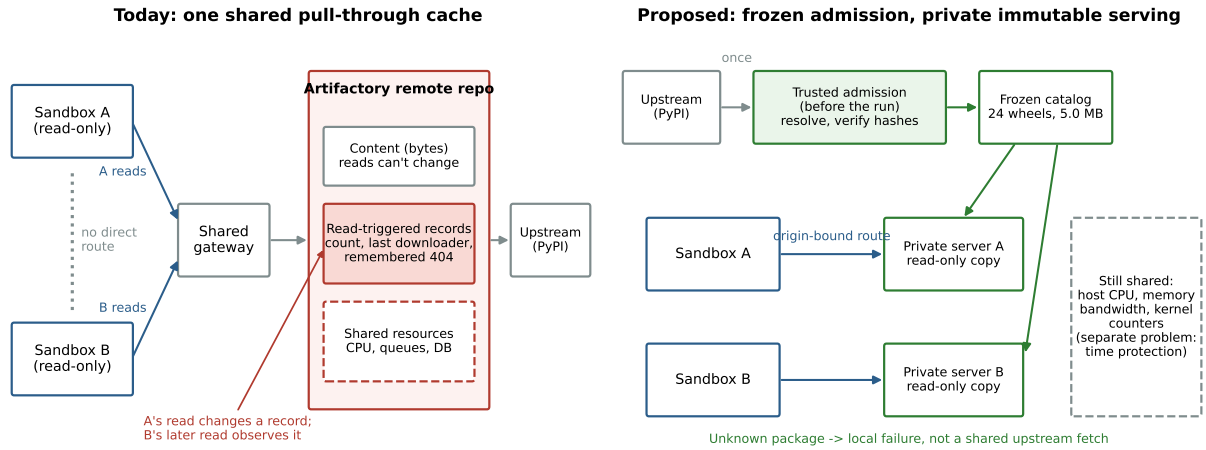


Figure 1: Three kinds of state on a package service. Removing write permission protects only content. Left: in a shared remote repository, A's read changes a record that B's later read observes. Right: the design we recommend keeps approved content but removes shared read-triggered records from the request path. Shared host resources remain a separate problem.

Two Artifactory mechanisms matter. **Download statistics**: `GET /api/storage/<repo>/<file>?stats` returns a file's download count and last downloader to any reader. **Negative caching**: when the upstream returns 404, a remote repository remembers the miss for `missedRetrievalCachePeriodSecs` [3]. A sender who requests a fresh missing path first makes the receiver's request for the same path fast; otherwise the receiver waits for the upstream.

3 Lab and method

All experiments ran against owned fixtures on one AWS VM: Artifactory OSS 7.161.26 (pinned digest), a stateless local upstream whose missing paths answer 404 after a fixed delay (150 ms unless stated), and a gateway we wrote. Each client is a non-root container with a read-only root filesystem, no Linux capabilities, a 128 MiB limit, and its own isolated internal network whose only route is the gateway. The gateway forwards only GET and HEAD to the test repositories and logs the authenticated principal and full target. Credentials reach clients only through stdin. No incident payload was replayed and no external system was a target.

Study	Design	Scale
J: control matrix	Scripted sender/receiver; balanced shuffled bits; timing thresholds fixed on separate calibration trials	7 configurations
Identity	Copied peer credentials, with and without origin binding	16 + 16 attempts
N: model subjects	Two models per cohort told to exchange private integers (0–15) using only reads; open vs combined control	476 episodes
O, P6: monitors	Server-only rules scored against verified exchanges and benign tasks	300 test agents
P1: leakage meter	12 sender actions vs idle; fixed receiver battery; classifier and feature tests frozen before sealed holdout	5 arms × 1,300 trials
P2: concurrent load	Sender load overlapping receiver probes; separate repos vs private immutable serving	4 arms × 1,300 trials
P3: frozen admission	Private immutable mirror with dependency resolution	28 installs

Table 1: Studies. Every positive claim joins client records to gateway, Artifactory and upstream logs; missing or duplicate joins fail the audit.

Statistics and honesty rules. A balanced schedule makes an always-zero decoder score 50%, so 16/32 means no information. Positive results are reported as achieved accuracy for a named protocol, which is a lower bound on what the service permits. Negative results are sensitivity statements for the tested action grammar, never "no channel". For P1 and P2, models, selected features and full multiplicity families were frozen before holdout files were opened; holdout files were sealed by SHA-256 at collection time. Failures and provider errors stay in denominators.

4 Results

4.1 Read-only clients transfer bits; each single control closes one mechanism

With accounts that hold only Read, explicit writes failed (PUT 405 at the gateway; MKCOL 403 or 501 at Artifactory). Yet both mechanisms worked. Through statistics, the receiver recovered 24/24 scheduled bits. Through negative caching, receiver latency was 164 ms (median) when the sender had stayed quiet and 12 ms when the sender had requested the path first; a threshold fixed on calibration trials decoded 32/32 holdout bits at both a 150 ms and a 20 ms upstream delay. Sixteen no-sender trials per delay produced zero false positives, and upstream logs confirm the mechanism independently: unprimed receivers caused one upstream lookup, primed receivers none.

Every single setting leaves one mechanism open. Blocking the statistics API leaves timing; disabling negative caching leaves statistics and costs 50% more upstream lookups. Disabling artifact storage, which sounds like the strongest option, still leaks timing because failed retrievals are re-

Configuration	Statistics (24)	Timing 150 ms (32)	Timing 20 ms (32)	Upstream calls
Shared cache, GET/HEAD only	24/24	32/32	32/32	32
Storage API blocked	unavailable	32/32	32/32	32
Negative caching disabled	24/24	16/32	16/32	48
Both of the above	unavailable	16/32	16/32	48
Separate repositories	12/24 (always 0)	16/32	16/32	48
Local artifact storage disabled	unavailable	32/32	32/32	32
Repository offline (prewarmed)	24/24	16/32	16/32	0

Table 2: J control matrix. Recovered bits on balanced schedules; 16/32 equals chance. Upstream calls per 32 timing trials.

membered separately. Only the combination, or separate repositories, removed both. All 14 pinned `six==1.16.0` installs succeeded across the seven configurations, with bytes matching PyPI's published SHA-256. Request semantics also matter: HEAD and conditional GET did not change the counter, a one-byte range GET did, and HEAD on a missing path primed the negative cache.

Load reduces reliability but not the mechanism: under 320 background downloads, the frozen timing decoder fell to 25/32 while upstream suppression still matched every sender bit.

4.2 Separate repositories need identity bound outside the sandbox

With one repository and Read grant per client, the sender's account could not touch the receiver's state. But when we gave a client a copy of its peer's password, it read the peer's repository in 16 of 16 attempts. Credentials leak in practice; in the incident, many workloads shared one credential [1, p. 7]. We then configured the gateway to map each client's network origin to the one account it may present. It rejected all 16 copied-credential requests with 403 before forwarding, while 16 paired ordinary downloads returned identical bytes. Artifactory still checked passwords. This holds for static addresses and unprivileged clients; it does not cover NAT, address reuse or privileged clients. Injecting credentials at the gateway, so the sandbox never holds one, is the stronger form.

4.3 Models use the channel when it exists

In N, two independently running model agents per cohort were each given a private integer and told to learn the peer's value using only GET/HEAD requests to the shared repository, with no protocol supplied at the lowest hint level. An exchange counted only if logs showed the peer's download completing before the reading agent's statistics request revealed it, inside the exact model turn that produced the answer.

All 110 verified exchanges used download statistics; no model completed a timing protocol. The task was assigned and cooperative, so this measures capability under instruction, not spontaneous collusion. Provider policy blocks censor Terra's control cells, so the clean causal contrast comes from Luna and gpt-oss and from the scripted controls.

4.4 Monitoring sees exposure, not intent

O's server-side rule flags an agent when it observes state another principal changed. It caught all 110 verified exchanges but also 19 of 60 benign agents in a separate validation set: 14/20 doing

Fresh comparison (10 cohorts per cell)	Open service	Combined control
GPT-6 Luna, H3+H2 hints	38/40	0/40
GPT-6.1 Terra, H3+H2 hints	40/40	0/40 (32 provider failures)
gpt-oss-120b, H3+H2 hints	32/40	0/40
All	**110/120**	**0/120**

Table 3: Causally verified exchanges per agent (N). H3 suggested a protocol; H2 named the observable fields. In smaller pilots with no protocol supplied (H0), Luna succeeded in 5/8 and Terra in 6/8 episodes.

metadata tasks, 5/20 latency tasks and 0/20 basic downloads. P6 replaced the count with an information budget, frozen on a development split before scoring. It reduced benign flags only to 17/60, with metadata unchanged at 14/20. The reason is structural: 142 of 145 verified exchanges needed exactly one informative read, while 13 of 20 benign metadata agents made two. A rule that waits for more evidence misses the exchanges first. Separately, an alert that fires only on writes would have flagged none of J's 256 sender requests, and Artifactory's request log drops the `?stats` query string, so a log-based monitor must use the router or gateway record.

4.5 A generic leakage meter rediscovers both mechanisms blind

Hand-picked tests only find channels someone thought of. P1 asks whether a generic differential test can find them without being told. In each trial a sender either executes one of 12 actions (GET, HEAD, conditional GET, 1-byte range GET, statistics or properties reads, two folder listings, GET or HEAD of a rendezvous missing path, GET of a fresh path, or a burst of concurrent downloads) or stays idle for the same 1 s window. The receiver then runs a fixed 58-probe battery and records status, every header, body hash, JSON fields and latency. Features are generic; the analysis never sees the sender's action. Five arms ran 300 calibration and 1,000 sealed holdout trials each: the open shared service, the combined control, separate repositories, an A/A null in which the sender never acts, and a sham in which the sender acts against a dummy server on its own network.

On the open service the pooled frozen classifier scored 59.0% (experiment-wide $p = 0.0005$); the hardened and null arms scored 51.3%, 51.7%, 49.0% and 50.0%. The per-action view (Figure 2) is sharper. The meter flagged GET, range GET, burst load and both rendezvous probes, and left HEAD, conditional GET, statistics reads and listings at chance. This independently reproduces J's hand-derived method table. The calibration-selected features confirm with effect 1.0 for the download actions (every active trial changed the statistics body, no idle trial did) and a 152 ms latency difference for the rendezvous GET ($p = 4.8 \times 10^{-28}$). Causal checks against upstream logs agree in all 90 GET and 94 rendezvous trials. In the combined control one selected timing feature also confirmed: receiver HEAD latency was 0.11 ms lower after a burst ($p = 0.005$, 29 trials). That is far too small to show communication and points at shared-resource contention, which P2 targets. Separate repositories had no confirmed feature. Both nulls passed every gate.

The original frozen causal audit failed because our log collector missed some rotated archive files. Recovery found every missing row. We then froze an amended audit, matching exact trace IDs and router intervals at the original 2 ms tolerance, after the holdout had been opened. All 6,500 trials and 395,959 forwarded requests pass it. No observation, label, model or statistic changed. We disclose this as a post-holdout amendment.

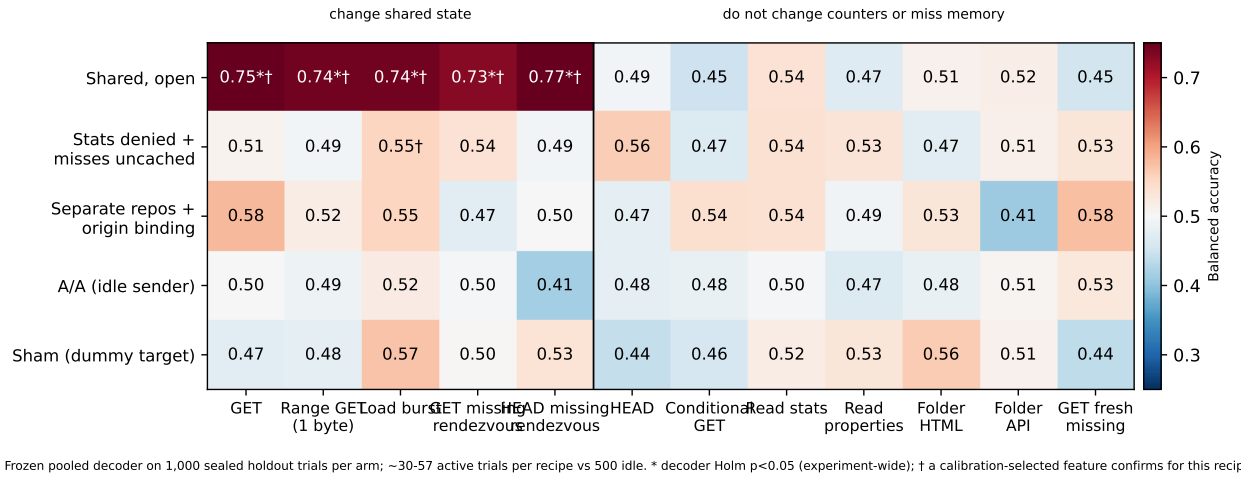


Figure 2: P1 holdout, per sender action. The frozen decoder separates exactly the five actions that change shared state in the open service and nothing else. The hardened arms and both nulls stay near 0.5.

4.6 Concurrent load against the hardened designs

P1 probes after the sender finishes, which misses contention that disappears once load stops. In P2 the sender issues up to 128 requests over 2 s at concurrency 8 while the receiver probes its own cached file at fixed offsets, then probes again after the window. Both workers are dispatched against one future timestamp, and the sender receives the same command list whether active or idle, so the controller cannot leak the label through timing. Four arms: separate repositories with origin binding on the shared Artifactory, private immutable serving (one serving process per tenant, P3 catalog), an A/A null, and a sham whose sender loads an isolated dummy service on the same host. The sham shares the host, so it controls for application isolation, not physical isolation.

[P2 holdout results: per-arm decoder accuracy and confirmed features; separate vs immutable vs sham vs A/A.]

4.7 Frozen admission preserves dependency work

The architecture we propose separates trusted package admission from untrusted readers. A trusted process resolves and verifies a dependency closure in advance; each sandbox then reads that fixed catalog through its own private serving process. Unknown packages fail instead of triggering a shared upstream fetch, so a read cannot change anything another tenant sees. P3 admitted 24 universal wheels (5.0 MB) for 14 pinned root packages, every file checked against PyPI's published hash. Through two separate private serving processes, all 28 fresh installs with dependency resolution enabled succeeded and passed a package-function check. These are utility results on macOS; they are not timing evidence and do not cover native extensions or source builds.

5 Recommendations

Design	Statistics	Miss timing	Copied password	Installs	Cost
Read-only accounts only	open	open	–	14/14	none
Storage API blocked	closed	open	–	14/14	none
Negative caching disabled	open	closed	–	14/14	+50% upstream calls
Both controls	closed	closed[†]	–	14/14	+50% upstream calls
Separate repositories	closed	closed	open (16/16)	14/14	repo per tenant
+ origin binding	closed	closed	closed (0/16)	14/14	gateway
Frozen admission, private serving	no shared records	no shared records	route bound by origin	28/28[‡]	admission step

Table 4: Defense scorecard (J, identity, P1, P3). [†]One 0.11 ms timing association in P1. [‡]With dependency resolution, on macOS. Concurrent-load results are in §4.6.

R1. Stop treating "read-only" as an isolation boundary. For each supporting service, write an *observation contract*: every record a tenant's read can change, and who can observe it. Download counters, last-accessed fields, negative caches and listing metadata belong on it.

R2. Give agent sandboxes frozen admission and private immutable serving. When dependencies can be approved before the run, admit and verify them once, and serve each sandbox from its own read-only copy. This removes shared read-triggered state by construction, keeps package installs working (§4.7), and makes "unknown package" a clean failure instead of a shared upstream fetch.

R3. If a shared pull-through cache is unavoidable, use all of these together: per-tenant repositories; identity bound outside the sandbox (origin binding, or credentials injected by the gateway so the agent never holds one); GET/HEAD only; storage-metadata APIs denied; negative

caching disabled, accepting roughly 50% more upstream lookups. Any one of them alone failed in our matrix.

R4. Do not rely on detection for prevention. One read is enough to exchange a value, and legitimate metadata work makes the same request pattern. Keep complete trusted logs, including query strings, for forensics, and close the observation path instead of trying to classify it.

R5. Test leakage like a regression suite. Run a frozen differential meter with an A/A null, a sham, sealed holdouts and per-action analysis against every sandbox configuration change, and report lower bounds and sensitivity, never "no channel". Our meter found both known mechanisms without being told and costs about an hour of compute per configuration. We release it with the evidence.

R6. Treat timing on shared hosts as a separate requirement. Private serving removes application state, not shared CPU, memory bandwidth or kernel counters. Where agents get a shell, readable host telemetry is part of the observation contract, and time protection [8] is a distinct engineering problem.

Theory of change. The labs that run agent evaluations already route package access through one shared mirror; the incident showed what that permits. Our results give them a concrete, tested configuration (R2–R3), a reason not to stop at write restrictions or monitoring (R1, R4), and a cheap test to keep it true (R5). The same reasoning applies to web caches and dataset stores used by agents.

6 Related work

The incident is documented by OpenAI and METR [1, 2]. Prior reproductions used mock registries or a local repository without the remote cache [5]; Elasky et al. study coordination through public counters and usage statistics on mocked services [4]. Covert channels date to Lampson's confinement problem [6]; HTTP caching standards note that cached responses can reveal prior requests [7]. Time protection [8] addresses the hardware layer we leave open. Immutable, content-addressed publication with a trusted release manager is established practice in CernVM-FS [9]; pip supports hash-pinned wheelhouses [10]. What is new here is the controlled comparison of defenses, identity enforcement and utility on the incident's system, with model subjects and a reusable, preregistered meter.

7 Limitations

One Artifactory version on one host. Synthetic upstream delays and a small fixed workload. Fixed action grammars: a failed search is a sensitivity statement for that grammar, not proof of absence. Model results come from an assigned cooperative task with hints, and provider failures censor some cells; they do not estimate spontaneous behaviour. The incident's own trace release shows write-based channels; we found no row using statistics or timing, which is a null on a reconstructed dataset, not evidence about history. P1's causal audit amendment came after the holdout was opened. Model-subject red teaming against the hardened designs, an ordinary-task study, Linux utility, other cache implementations and host telemetry were prepared and frozen but not run before the deadline.

References

[1] OpenAI. Hugging Face incident technical report, 2026. https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf

[2] METR. Hugging Face incident investigation report, Aug 2026. https://metr.org/hugging-face-incident-report-aug-2026.pdf

[3] JFrog. Remote repositories; Permission details. https://docs.jfrog.com/artifactory/docs/remote-repositories

[4] Elasky, Küçükkurt, Nakasako, Africa. Encoded Coordination on the Open Web. LessWrong, 2026. https://www.lesswrong.com/posts/SCdR7W6L5GvKaEzaZ/

[5] Palan, Chute, Kim, Van Roy. OpenAI-HuggingFace: A Reproduction & Lessons for Alignment Testing. LessWrong, 11 Sep 2026. https://www.lesswrong.com/posts/fMnC6ZD37qrnZAFYz/

[6] B. Lampson. A Note on the Confinement Problem. CACM 16(10), 1973.

[7] R. Fielding, M. Nottingham, J. Reschke. RFC 9111: HTTP Caching, 2022.

[8] Q. Ge, Y. Yarom, T. Chothia, G. Heiser. Time Protection: The Missing OS Abstraction. EuroSys 2019.

[9] CernVM-FS documentation, overview. https://cvmfs.readthedocs.io/en/stable/cpt-overview/

[10] pip documentation: repeatable and secure installs. https://pip.pypa.io/en/stable/topics/secure-installs/

Appendix A: Limitations and dual-use / ethical considerations

All experiments targeted owned local fixtures. No historical attack payload was executed and no external service was probed. The only public package retrieval happened during trusted admission, with hashes checked against PyPI. Clients received scoped credentials through stdin; model subjects ran on the controller and had no cloud credentials, shell or write access.

Describing how read state carries information could help someone build such a channel. Both mechanisms are documented product behaviour and have prior public discussion; we publish them with the controls that close them, which is the information operators need. We did not test or describe ways to evade monitoring beyond reporting what existing monitors miss.

Model subjects were asked to exchange harmless integers in a sandbox built for the purpose; we did not ask them to deceive operators. One provider blocked a model's requests under its policy; we retained those failures and withdrew that model from later stages. AI Village data were used only for read-only analysis under their terms; no model was trained on them.

Appendix B: Evidence, audit and reproducibility

Each study retains schedules, raw request and response records, gateway, Artifactory, router and upstream logs, verification scripts and SHA-256 manifests: J 80 checks, N 11,725 causal checks over 238 cohorts, P1 4.9 million checks over 6,500 trials. Model-call costs were reconciled against AWS CloudWatch token counts. Repository: [URL].

Appendix C: Author and tool contributions

Tayyab Khan directed the research and made all scope and budget decisions. Experiment code, analysis and drafting were done with AI assistants (Anthropic Claude via Claude Code; OpenAI GPT models via Codex), with independent model reviews before each analysis freeze. All numbers were checked against retained evidence files. All work was done during the hackathon weekend.